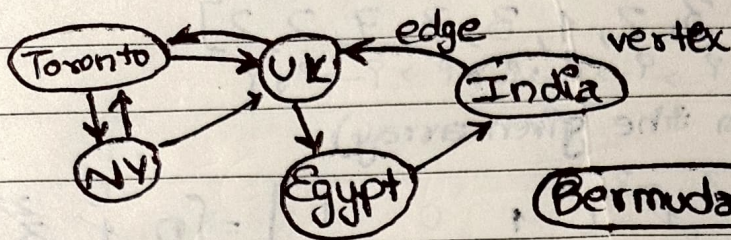


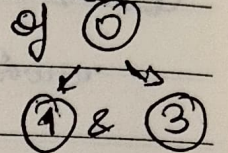
Graphs

Example:



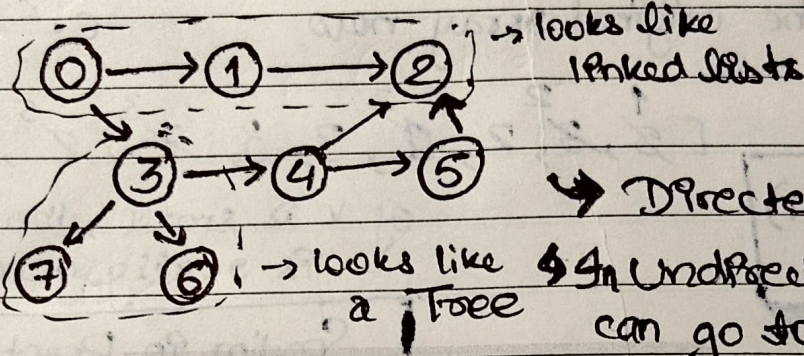
→ Not connected to any other vertex but still is a part of the Graph

Neighbours



$N=8$

vertices are commonly known as nodes



Directed graph

↳ Undirected graph we can go to & from a node to another.

↳ If there's a cycle in the graph then it is called a cyclic graph

Edge-List = $[[0,1], [1,2], [3,4], [3,6], [3,6], [3,7], [1,2], [4,5], [5,2]]$

↓ represents edges (can be directed or undirected)

↳ Edge list is really irritating to work with

Adjacency Matrix

	0	1	2	3	4	5	6	7
0	0	1	0	1	0	0	0	0
1	0	0	1	0	0	0	0	0
2	0	0	0	0	0	0	0	0
3	0	0	0	0	1	0	1	1
4	0	0	1	0	0	1	0	0
5	0	0	1	0	0	0	0	0
6	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0

To math problem with adjacency matrix is that we're storing a ton of information.

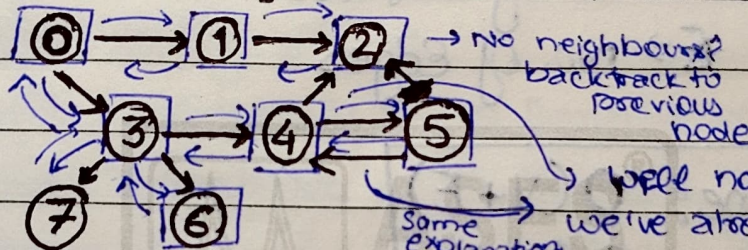
↳ If we don't have any connections - why do we have to loop over this whole row to figure that out.

Adjacency ListImplemented
using Hashmap
 $\hookrightarrow \{0: [1, 3], 1: [2], 2: [], 3: [4, 6, 7], 4: [2, 5], 5: [2], 6: [], 7: []\}$
Class Node:

Node.value

Node.neighbours = []

① & ③ are references to these nodes.

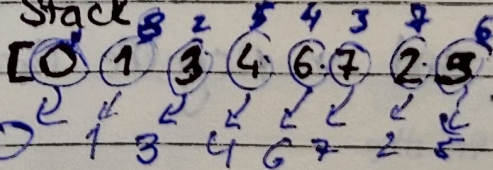
Recursive [Internally uses call stack] $\hookrightarrow$ for ① = [1, 3]DFS [Depth First Search] $\rightarrow$ Prioritizes depthvery important
for any
traversal ~~is~~ to
have a
seen set

Seen = (0 1 2 3 4 5 6 7)

which is going to be a hash set which stores the node we've been to

without the seen set we would've been trapped in the (4) $\leftrightarrow$ (5)cycle for
forever.Iterative DFS (stack)

while Stack



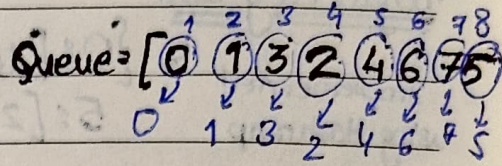
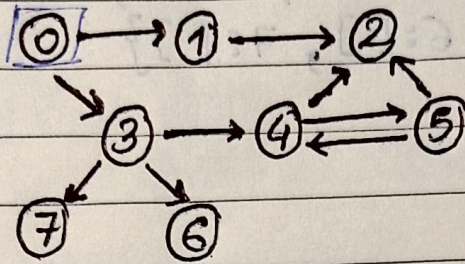
pop order

0
3
7
6
4
5
2
1

seen = (0 1 3 4 6 7 2 5)

BFS [Breadth First Search] (Queue)

~~Source = 0~~



pop order

0
1
3
2
4
6
7
5

seen = (0 1 3 2 4 6 7 5)

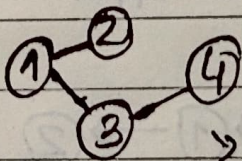
$V \rightarrow$ No. of vertices

$E \rightarrow$ No. of edges

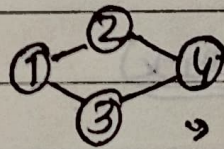
(using adjacency list)

Time: $O(V+E)$
Space: $O(V+E)$

Trees (In context of Graphs)



$\rightarrow$ A tree



$\rightarrow$ Not a tree

$\rightarrow$ Tree is called a connected and an Acyclic Graph.

for N vertices from anywhere in the graph you can get to anywhere else in the graph.

then $E = N - 1$ edges

Eg: $N = 4$
 $E = 3$